

Linear & Logistic Regression

The Mathematical Foundation of Machine Learning

■ REGRESSION MODELS

■ LEARNING OUTCOME

By the end of this module you will understand how linear and logistic regression work mathematically, implement them with sklearn, apply regularisation, and know when to use each model in practice.

■ Skills You Will Gain

- Derive and interpret the linear regression equation
- Understand cost function and gradient descent intuitively
- Apply Ridge (L2) and Lasso (L1) regularisation
- Build logistic regression for binary classification
- Interpret coefficients and odds ratios
- Evaluate models with RMSE, R-squared, and AUC-ROC

■ Regression is used daily in finance (risk models), healthcare (dosage), marketing (ROI), and engineering. Mastering it unlocks every other ML algorithm.

SECTION 1

Linear Regression — How It Works

The Linear Regression Equation

$y_{\text{hat}} = B_0 + B_1x_1 + B_2x_2 + \dots + B_nx_n$ Goal: Find coefficients B that minimise Mean Squared Error: $MSE = (1/n) * \sum((y_i - y_{\text{hat}_i})^2)$ Gradient Descent updates weights: $B = B - \alpha * dMSE/dB$ (α = learning rate)

Term	Meaning
Intercept (B_0)	Predicted y when all features = 0
Coefficient (B_i)	Change in y per 1-unit increase in x_i , all else equal
R-squared	Fraction of variance explained — $R^2=0.85$ means 85% of target variation is captured
RMSE	Root Mean Square Error — in same units as target, penalises large errors more
Residual	Actual minus Predicted — should be random and centred at zero
Multicollinearity	High correlation between predictors — makes coefficients unstable and hard to interpret

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso from sklearn.metrics import mean_squared_error, r2_score import numpy as np model = LinearRegression() model.fit(X_train, y_train) y_pred = model.predict(X_test) rmse = np.sqrt(mean_squared_error(y_test, y_pred)) r2 = r2_score(y_test, y_pred) print(f'RMSE: {rmse:.2f} | R2: {r2:.4f}') print(f'Intercept: {model.intercept_:.2f}') print('Coefficients:', dict(zip(X.columns, model.coef_))) # Regularised (penalises large coefficients) ridge = Ridge(alpha=1.0).fit(X_train, y_train) # L2: shrinks all towards zero lasso = Lasso(alpha=0.1).fit(X_train, y_train) # L1: drives irrelevant features to exactly zero
```

SECTION 2

Logistic Regression — Binary Classification

The Sigmoid Function

Logistic regression passes the linear combination through a sigmoid function: $\sigma(z) = 1 / (1 + e^{-z})$ --> output is always between 0 and 1 Interpretation: output = $P(y=1 | X)$ — the probability of the positive class Decision: predict class 1 if $\sigma(z) \geq 0.5$ (threshold can be adjusted)

Concept	Detail
Coefficient	Each unit increase in x_i multiplies odds by $\exp(B_i)$
Odds Ratio	$\exp(\text{coefficient})$ — e.g. $\exp(0.5)=1.65$ means 65% higher odds
Log-Loss	Cost function: $-[y*\log(p) + (1-y)*\log(1-p)]$ — lower is better
C parameter	Inverse of regularisation strength — higher C = less regularisation
max_iter	Maximum gradient descent iterations — increase if model fails to converge

```

from sklearn.linear_model import LogisticRegression from sklearn.metrics import
classification_report, roc_auc_score model = LogisticRegression(max_iter=1000, C=1.0) # C =
1/lambda model.fit(X_train_scaled, y_train) y_pred = model.predict(X_test_scaled) y_proba =
model.predict_proba(X_test_scaled)[: , 1] print(classification_report(y_test, y_pred))
print(f'AUC-ROC: {roc_auc_score(y_test, y_proba):.4f}') # Interpret coefficients import pandas as
pd, numpy as np coefs = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_[0],
'Odds_Ratio': np.exp(model.coef_[0]) }) print(coefs.sort_values('Coefficient', ascending=False))

```

SECTION 3

Regularisation: Ridge vs Lasso vs ElasticNet

Method	Penalty Added	Effect	Best For
Ridge (L2)	$\alpha \cdot \sum (B_i^2)$	Shrinks all coefficients — none to zero	Multicollinear features, keep all
Lasso (L1)	$\alpha \cdot \sum (B_i)$	Drives irrelevant features to EXACTLY zero	Feature selection, sparse model
ElasticNet	L1 + L2 combined	Groups correlated features together	High-dim with correlated features
No penalty	None	Can overfit with many features	Simple problems, few features

■ PRACTICE Q & A

- Q1.** What does R-squared = 0.72 mean for a linear regression?
- A: 72% of the variance in the target variable is explained by the model's features. The remaining 28% is unexplained noise.*
- Q2.** When would you use Lasso instead of Ridge?
- A: When you suspect many features are irrelevant. Lasso shrinks irrelevant coefficients to exactly zero, performing automatic feature selection.*
- Q3.** What does the logistic regression coefficient of 0.6 for Age mean?
- A: Each 1-year increase in Age multiplies the odds of the positive outcome by $e^{0.6} = 1.82$ (82% higher odds).*
- Q4.** Why scale features before logistic regression?
- A: Gradient descent converges faster when features are on similar scales. Unscaled features with different magnitudes cause slow, unstable training.*
- Q5.** What is the decision threshold in logistic regression and when would you change it?
- A: Default is 0.5. Lower it (e.g. 0.3) to catch more positives (higher recall, lower precision) — useful for fraud/disease detection where missing a case is costly.*

■ Regression Cheat Sheet	
<code>LinearRegression()</code>	Standard OLS regression
<code>Ridge(alpha=1.0)</code>	L2 regularisation – shrink all
<code>Lasso(alpha=0.1)</code>	L1 + feature selection
<code>LogisticRegression(C=1.0)</code>	Binary classification
<code>model.coef_</code>	Feature coefficients
<code>model.intercept_</code>	Bias/intercept term
<code>r2_score(y, yp)</code>	R-squared – variance explained
<code>mean_squared_error(y, yp)</code>	MSE loss
<code>roc_auc_score(y, proba)</code>	AUC-ROC score
<code>predict_proba(X)[: , 1]</code>	Probability of class 1
<code>sigma(z) = 1/(1+e^{-z})</code>	Sigmoid activation

Odds Ratio = $\exp(\text{coef})$	Interpret log-odds
----------------------------------	--------------------